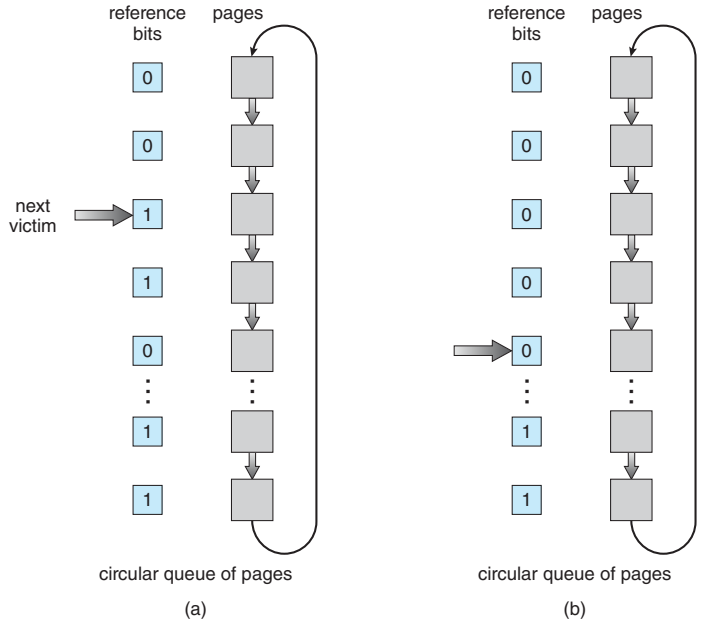
۳- (1, 0) اخیراً استفاده شده، اما دست‌نخورده است شاید به زودی باز هم مورد استفاده قرار بگیرد.
۴- (1, 1) اخیراً استفاده داشته و تغییر یافته است شاید به زودی باز هم مورد استفاده قرار بگیرد و پیش از جایگزینی لازم است که صفحه در حافظه‌ی ثانویه نوشته شود.



شکل ۱۰-۱۷ الگوریتم جایگزینی صفحه‌ی شانس دوم (ساعت).

هر صفحه در یکی از این رده‌ها قرار می‌گیرد. زمانی که قرار است جایگزینی صفحه صورت بگیرد، از همان طرح الگوریتم ساعت استفاده می‌کنیم؛ اما به جای این که بررسی کنیم صفحه‌ای که به آن اشاره داریم بیت مراجعه‌اش ۱ است یا خیر، رده‌ای را که صفحه به آن تعلق دارد بررسی می‌کنیم. اولین صفحه‌ای را که در پایین‌ترین رده‌ی ناتهی یافتیم برای جایگزینی انتخاب می‌کنیم. توجه داشته باشید که امکان دارد پیش از این که صفحه‌ای را برای جایگزینی پیدا کنیم صف چرخشی را چندین بار پیمایش کنیم. تفاوت اصلی میان این الگوریتم و الگوریتم ساده‌تر ساعت آن است که در اینجا، برتری را به صفحاتی می‌دهیم که تغییر یافته‌اند تا دفعات مورد نیاز I/O را کاهش دهیم.

۱۰-۴-۶ جایگزینی صفحه‌ی مبتنی بر شمارش

الگوریتم‌های فراوان دیگری هم هستند که می‌توانند برای جایگزینی صفحه استفاده شوند. برای مثال، می‌توانیم شمارنده‌ای برای تعداد مراجعات هر یک از صفحات نگه‌داری کنیم و دو طرح زیر را برای جایگزینی صفحه به کار ببریم.

- الگوریتم جایگزینی صفحه با **حداقل دفعات استفاده**[1] (**LFU**) لازم دارد که صفحه‌ای با کوچک‌ترین شمارش جایگزین شود. دلیل این گزینش آن است که صفحه‌ای که استفاده‌ی اکیدی دارد شمارش مراجعات بالایی خواهد داشت و لذا انتخاب نمی‌شود. با این وجود، مسأله‌ای که در اینجا پیش می‌آید این است که، امکان دارد صفحه‌ای در آغاز اجرای پردازه کاربرد وزینی داشته باشد، اما در ادامه هرگز مورد استفاده واقع نشود. چون استفاده‌ی سنگینی داشته، شمارش بزرگ‌تری خواهد داشت، هرچند که دیگر مورد نیاز نیست، باز هم در حافظه باقی می‌ماند. یک راه‌حل آن است که شمارنده‌ها را در فواصل زمانی منظمی به سمت راست (۱ بیت) شیفت بدهیم تا یک میانگین شمارش استفاده‌ی نمایی میرا شکل بگیرد.
- الگوریتم جایگزینی صفحه با **حداکثر دفعات استفاده**[2] (**MFU**)، این استدلال را دارد که صفحه‌ای که کوچکترین شمارنده را دارد احتمالاً به تازگی به حافظه آورده شده و باز هم مورد استفاده قرار خواهد گرفت.

همان‌گونه که ممکن است انتظار داشته باشید هیچ یک از جایگزینی‌های MFU و یا LFU رواج ندارند. پیاده‌سازی این الگوریتم‌ها گران است و آنها جایگزینی OPT را به خوبی تقریب نمی‌زنند.

۱۰-۴-۷ الگوریتم‌های بافرسازی صفحه

علاوه بر الگوریتم جایگزینی صفحه‌ی خاص غالباً از روال‌های دیگری نیز استفاده می‌شود. برای مثال، سیستم‌های زیادی هستند که انباری از قاب‌های آزاد را نگه می‌دارند. لحظه‌ای که یک خطای صفحه رخ می‌دهد، باز هم مثل سابق یک قاب قربانی انتخاب می‌شود. با این وجود، پیش از آنکه قربانی در دیسک نوشته شود، صفحه‌ی مطلوب به داخل قاب آزادی از انبار خوانده می‌شود. این روال موجب می‌شود که پردازه کار خود را بدون انتظار نوشتن صفحه‌ی قربانی در دیسک در اسرع وقت شروع کند. بعدها، زمانی که قربانی در دیسک نوشته شد، قاب آن به انبار قاب آزاد افزوده می‌شود.

یک تعمیم از این ایده آن است که لیستی از صفحات تغییریافته نگه‌داری شود. هر زمان که دستگاه صفحه‌بندی بیکار است، صفحه‌ی تغییریافته‌ای انتخاب شده و در حافظه‌ی ثانویه نوشته می‌شود. سپس بیت تغییر آن صفحه پاک می‌شود. این طرح احتمال تمیزی صفحه را در موقع انتخاب آن برای جایگزینی افزایش می‌دهد و لذا در جایگزینی نیازی برای دوباره‌نویسی در دیسک نخواهد داشت.

اصلاحیه‌ی دیگری که برای این روش مطرح است آن است که انباری از قاب‌های آزاد نگه‌داری کنیم اما، به خاطر بسپاریم که در هر قاب چه صفحه‌ای قرار دارد. از آنجایی که محتویات قاب با نوشتن در حافظه‌ی ثانویه تغییر نمی‌یابد، اگر پیش از آنکه این قاب دوباره مورد استفاده قرار بگیرد مورد نیاز باشد، می‌توان صفحه‌ی قدیمی را باز هم مستقیماً از لیست قاب آزاد مورد استفاده قرار داد. در این حالت هیچ‌گونه I/O لازم نیست. لحظه‌ای که یک خطای صفحه رخ می‌دهد،

1. Least Frequently Used
2. Most Frequently Used

ابتدا بررسی می‌کنیم که آیا صفحه‌ی مطلوب همچنان در انبار قاب آزاد موجود است. اگر نبود، بایستی یک قاب آزاد انتخاب کرده و صفحه را به داخل آن بخوانیم.

برخی نسخه‌های سیستم یونیکس از این روش به همراه الگوریتم شانس دوم استفاده می‌کنند. بودن این راهکار در کنار هر یک از الگوریتم‌های جایگزینی دیگر می‌تواند مفید باشد، چون جریمه‌ای را که در صورت گزینش اشتباهی یک صفحه‌ی قربانی عاید سیستم می‌شود، کاهش می‌دهد. ما این‌ها—و سایر—اصلاحات را در بخش ۱۰-۵-۳ بیان می‌کنیم.

۱۰-۴-۸ برنامه‌های کاربردی و جایگزینی صفحه

در شرایط خاصی، آن دسته از برنامه‌های کاربردی که داده‌هایشان را از طریق حافظه‌ی مجازی سیستم‌عامل دسترسی می‌نمایند بدتر عمل می‌کنند و اگر سیستم‌عامل بافرسازی را اصلاً فراهم نمی‌کرد بهتر بود. مثال نوعی این مورد پایگاه‌داده است که مدیریت حافظه و بافرسازی I/O خاص خود را فراهم می‌کند. این قبیل برنامه‌های کاربردی چگونگی استفاده از حافظه و حافظه‌ی ثانویه‌ی خود را بهتر از آنچه که سیستم‌عامل با پیاده‌سازی الگوریتم‌های با کاربرد همه‌منظوره فراهم می‌کند، می‌فهمند. به‌علاوه، اگر سیستم‌عامل بافرسازی I/O را انجام دهد و همین کار را برنامه‌ی کاربردی نیز انجام بدهد، در این صورت دو برابر میزان مورد نیاز حافظه برای یک مجموعه از I/O مصرف می‌شود.

در مثالی دیگر، انبارهای داده[1] به کرات انبوهی از داده‌ها را به ترتیب از حافظه‌ی ثانویه خوانده و به دنبال آن یک سری محاسبات و نوشتنی‌ها دارند. الگوریتم LRU صفحات قدیمی را حذف کرده و صفحات جدید را نگه می‌دارد، در حالی که، برنامه‌ی کاربردی، احتمالاً صفحات قدیمی‌تر را بیشتر از صفحات جدید خواهد خواند (چرا که، خواندن خود را باز هم به ترتیب آغاز خواهد کرد). در اینجا، استفاده از MFU به واقع کارآمدتر از LRU خواهد بود.

به خاطر این قبیل مسائل، برخی سیستم‌عامل‌ها به برنامه‌های خاصی این توانایی را می‌دهند که یک پارتیشن حافظه‌ی ثانویه را به صورت آرایه‌ی ترتیبی بزرگی از بلاک‌های منطقی بدون حضور هیچ یک از ساختمان‌داده‌های سیستم‌فایل استفاده نمایند. بعضاً به این آرایه، **دیسک خام** گفته می‌شود و I/O این آرایه، در اصطلاح، I/O خام نامیده می‌شود. I/O خام، تمامی سرویس‌های سیستم‌فایل را از قبیل صفحه‌بندی نیازی I/O فایل، قفل‌گذاری فایل، پیش‌واکشی، تخصیص فضا، اسامی فایل و دایرکتوری‌ها دور می‌زند. توجه داشته باشید که اگرچه برنامه‌های کاربردی خاصی هستند که با پیاده‌سازی سرویس‌های حافظه‌ی تک‌منظوره‌ی خاص خودشان کارآمدی بیشتری دارند، بیشتر برنامه‌های کاربردی در زمانی که از سرویس‌های عادی سیستم‌فایل استفاده می‌کنند عملیات بهتری دارند.

۱۰-۵ تخصیص قاب‌ها

باز به مسأله‌ی تخصیص قاب بر می‌گردیم. چگونه حجم ثابتی از حافظه‌ی آزاد را مابین

1. Data warehouses